

TASK-1

```
siddu@DESKTOP-5CRM05P:~$ mkdir week3
siddu@DESKTOP-5CRM05P:~$ cd week3
siddu@DESKTOP-5CRM05P:~/week3$ nano task1.c
siddu@DESKTOP-5CRM05P:~/week3$ gcc task1.c -o task1
siddu@DESKTOP-5CRM05P:~/week3$ ./task1
mysh> hello
You entered: hello
mysh> exit
Exiting... Goodbye!
```

```
siddu@DESKTOP-5CRM05P:~/week3$ cat task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    while (1) {
        write(STDOUT_FILENO, "mysh> ", 6);

        bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);

        if (bytes_read <= 0) {
            write(STDOUT_FILENO, "\nExiting...\n", 12);
            break;
        }

        buffer[bytes_read] = '\0';

        if (buffer[bytes_read - 1] == '\n') {
            buffer[bytes_read - 1] = '\0';
        }

        if (strcmp(buffer, "exit") == 0) {
            write(STDOUT_FILENO, "Exiting... Goodbye!\n", 20);
            break;
        }

        if (strlen(buffer) > 0) {
            write(STDOUT_FILENO, "You entered: ", 13);
            write(STDOUT_FILENO, buffer, strlen(buffer));
            write(STDOUT_FILENO, "\n", 1);
        }
    }

    return 0;
}
```

```
siddu@DESKTOP-5CRM05P:~/week3$ █
```

TASK-2

```
siddu@DESKTOP-5CRM05P:~$ cd week3
siddu@DESKTOP-5CRM05P:~/week3$ nano File1.txt
siddu@DESKTOP-5CRM05P:~/week3$ nano task2.c
siddu@DESKTOP-5CRM05P:~/week3$ gcc task2.c -o task2
./task2
File copied successfully!
siddu@DESKTOP-5CRM05P:~/week3$ cat File2.txt
Hello! Testing system call file copy for Task 2.
```

```
siddu@DESKTOP-5CRM05P:~/week3$ cat File2.txt
Hello! Testing system call file copy for Task 2.
```

```
siddu@DESKTOP-5CRM05P:~/week3$ cat -n task2.c
```

```
1  #include <fcntl.h>
2  #include <unistd.h>
3  #include <stdio.h>
4
5  #define BUFFER_SIZE 1024
6
7  int main() {
8      int fd_src = open("File1.txt", O_RDONLY);
9      if (fd_src < 0) {
10         perror("Error opening File1.txt");
11         return 1;
12     }
13
14     int fd_dest = open("File2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
15     if (fd_dest < 0) {
16         perror("Error creating File2.txt");
17         close(fd_src);
18         return 1;
19     }
20     char buffer[BUFFER_SIZE];
21     ssize_t bytes_read, bytes_written;
22
23     while ((bytes_read = read(fd_src, buffer, sizeof(buffer))) > 0) {
24         bytes_written = write(fd_dest, buffer, bytes_read);
25         if (bytes_written != bytes_read) {
26             perror("Error writing to File2.txt");
27             break;
28         }
29     }
30
31     close(fd_src);
32     close(fd_dest);
33     write(STDOUT_FILENO, "File copied successfully!\n", 26);
34     return 0;
35 }
36
```

```
siddu@DESKTOP-5CRM05P:~/week3$
```

TASK-3

```
siddu@DESKTOP-5CRM05P:~$ nano task3.c
siddu@DESKTOP-5CRM05P:~$ cd ~/week3
siddu@DESKTOP-5CRM05P:~/week3$ nano task3.c
siddu@DESKTOP-5CRM05P:~/week3$ gcc task3.c -o task3
siddu@DESKTOP-5CRM05P:~/week3$ ./task3
mysh> hello
Command: hello

mysh> exit
siddu@DESKTOP-5CRM05P:~/week3$
```

```
siddu@DESKTOP-5CRM05P:~/week3$ cat -n task3.c
```

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <termios.h>
6
7  #define MAX 1024
8
9  int main() {
10     struct termios oldt, newt;
11     char buffer[MAX];
12     char ch;
13
14     tcgetattr(STDIN_FILENO, &oldt);
15     newt = oldt;
16
17     newt.c_lflag &= ~(ICANON | ECHO);
18     tcsetattr(STDIN_FILENO, TCSANOW, &newt);
19
20     while (1) {
21         write(STDOUT_FILENO, "\nmysh> ", 7);
22         int index = 0;
23
24         while (1) {
25             if (read(STDIN_FILENO, &ch, 1) <= 0) break;
26
27             if (ch == '\n' || ch == '\r') {
28                 buffer[index] = '\0';
29                 write(STDOUT_FILENO, "\n", 1);
30                 break;
31             }
32             else if ((ch == 127 || ch == 8) && index > 0) {
33                 index--;
34                 write(STDOUT_FILENO, "\b \b", 3);
35             }
36             else if (index < MAX - 1 && ch >= 32 && ch <= 126) {
37                 buffer[index++] = ch;
38                 write(STDOUT_FILENO, &ch, 1);
39             }
40         }
41
42         if (strcmp(buffer, "exit") == 0) {
43             break;
44         }
45     }
46 }
```

```
siddu@DESKTOP-5CRM05P:~/week3$ █
```

```
41     }
42
43     if (strcmp(buffer, "exit") == 0) {
44         break;
45     }
46
47     write(STDOUT_FILENO, "Command: ", 9);
48     write(STDOUT_FILENO, buffer, strlen(buffer));
49     write(STDOUT_FILENO, "\n", 1);
50 }
51
52 tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
53 return 0;
54 }
```

siddu@DESKTOP-5CRM05P:~/week3\$ █

TASK-4

```
siddu@DESKTOP-5CRM05P:~$ cd ~/week3
siddu@DESKTOP-5CRM05P:~/week3$ nano task4.c
siddu@DESKTOP-5CRM05P:~/week3$ gcc task4.c -o task4
siddu@DESKTOP-5CRM05P:~/week3$ ./task4
Mode: "r"
File1.txt  ExsCo1  ExsCo2  ExsCo3  ExsCo4
Mode: "w"
File2.txt  TasCo1  TasCo2  TasCo3  TasCo4

Mode: "a"
Appending data to File2.txt

Mode: "r+"
Reading and modifying File1.txt
```

```
siddu@DESKTOP-5CRM05P:~/week3$ cat -n task4.c
```

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <termios.h>
6  #include <sys/wait.h>
7
8  #define MAX_HISTORY 10
9  #define MAX_CMD 256
10
11 char history[MAX_HISTORY][MAX_CMD];
12 int history_count = 0;
13
14 int main() {
15     struct termios oldt, newt;
16     tcgetattr(STDIN_FILENO, &oldt);
17     newt = oldt;
18     newt.c_lflag &= ~(ICANON | ECHO);
19
20     while (1) {
21         tcsetattr(STDIN_FILENO, TCSANOW, &newt);
22         write(STDOUT_FILENO, "mysh> ", 7);
23
24         char buffer[MAX_CMD] = {0};
25         int index = 0, hist_idx = history_count;
26
27         while (1) {
28             char ch;
29             if (read(STDIN_FILENO, &ch, 1) <= 0) break;
30             if (ch == 27) {
31                 char seq[2];
32                 if (read(STDIN_FILENO, &seq[0], 1) > 0 && read(STDIN_FILENO, &seq[1], 1) > 0) {
33                     if (seq[0] == '[') {
34                         if (seq[1] == 'A' && hist_idx > 0) {
35                             hist_idx--;
36                             while (index > 0) { write(STDOUT_FILENO, "\b \b", 3); index--; }
37                             strcpy(buffer, history[hist_idx]);
38                             index = strlen(buffer);
39                             write(STDOUT_FILENO, buffer, index);
40                         } else if (seq[1] == 'B' && hist_idx < history_count) {
41                             hist_idx++;
42                             while (index > 0) { write(STDOUT_FILENO, "\b \b", 3); index--; }
43                             if (hist_idx == history_count) strcpy(buffer, history[hist_idx]);
```

```

44         if (hist_idx < history_count) strcpy(buffer, history[hist_idx]);
45         else buffer[0] = '\0';
46         index = strlen(buffer);
47         write(STDOUT_FILENO, buffer, index);
48     }
49 }
50 }
51 } else if (ch == '\n' || ch == '\r') {
52     buffer[index] = '\0';
53     write(STDOUT_FILENO, "\n", 1);
54     break;
55 } else if ((ch == 127 || ch == 8) && index > 0) {
56     index--;
57     write(STDOUT_FILENO, "\b \b", 3);
58 } else if (index < MAX_CMD - 1 && ch >= 32 && ch <= 126) {
59     buffer[index++] = ch;
60     write(STDOUT_FILENO, &ch, 1);
61 }
62 }
63
64 tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
65
66 if (strlen(buffer) == 0) continue;
67 if (strcmp(buffer, "exit") == 0) break;
68
69 if (history_count < MAX_HISTORY) {
70     strcpy(history[history_count++], buffer);
71 }
72
73 char *args[64];
74 int i = 0;
75 char *token = strtok(buffer, " ");
76 while (token != NULL) {
77     args[i++] = token;
78     token = strtok(NULL, " ");
79 }
80 args[i] = NULL;
81
82 pid_t pid = fork();
83 if (pid == 0) {
84     execvp(args[0], args);
85     perror("Execution failed");
86     exit(1);
87 } else if (pid > 0) {
88     wait(NULL);

```

```
71     }
72
73     char *args[64];
74     int i = 0;
75     char *token = strtok(buffer, " ");
76     while (token != NULL) {
77         args[i++] = token;
78         token = strtok(NULL, " ");
79     }
80     args[i] = NULL;
81
82     pid_t pid = fork();
83     if (pid == 0) {
84         execvp(args[0], args);
85         perror("Execution failed");
86         exit(1);
87     } else if (pid > 0) {
88         wait(NULL);
89     }
90 }
91
92 tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
93 return 0;
94 }
```

TASK-5

```
siddu@DESKTOP-5CRM05P:~/week3$ nano task5.c
siddu@DESKTOP-5CRM05P:~/week3$ gcc -g task5.c -o task5
siddu@DESKTOP-5CRM05P:~/week3$ valgrind --leak-check=full ./task5
==2403== Memcheck, a memory error detector
==2403== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2403== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2403== Command: ./task5
Enter text for mmap: Hello World
Input: Hello World
Updated string: Hello World - Memory Resized
Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Memory released successfully.

==2403== HEAP SUMMARY:
==2403==      in use at exit: 0 bytes in 0 blocks
==2403==    total heap usage: 6 allocs, 6 frees, 1,104 bytes allocated
==2403==
==2403== All heap blocks were freed -- no leaks are possible
==2403==
==2403== For lists of detected and suppressed errors, rerun with: -s
==2403== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
siddu@DESKTOP-5CRM05P:~/week3$ cat -n task5.c
```

```
1  #define _GNU_SOURCE
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <sys/mman.h>
7
8  struct Node {
9      int data;
10     struct Node* next;
11 };
12
13 int main() {
14     char *ptr = mmap(NULL, 100, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
15     if (ptr == MAP_FAILED) return 1;
16
17     write(STDOUT_FILENO, "Enter text for mmap: ", 21);
18     ssize_t bytes_read = read(STDIN_FILENO, ptr, 99);
19     if (bytes_read > 0 && ptr[bytes_read - 1] == '\n') bytes_read--;
20     ptr[bytes_read] = '\0';
21
22     write(STDOUT_FILENO, "Input: ", 7);
23     write(STDOUT_FILENO, ptr, strlen(ptr));
24     write(STDOUT_FILENO, "\n", 1);
25
26     ptr = mremap(ptr, 100, 200, MREMAP_MAYMOVE);
27     strcat(ptr, " - Memory Resized");
28
29     write(STDOUT_FILENO, "Updated string: ", 16);
30     write(STDOUT_FILENO, ptr, strlen(ptr));
31     write(STDOUT_FILENO, "\n", 1);
32
33     struct Node *head = NULL, *temp = NULL;
34     for (int i = 1; i <= 5; i++) {
35         struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
36         newNode->data = i * 10;
37         newNode->next = NULL;
38         if (!head) head = newNode;
39         else temp->next = newNode;
40         temp = newNode;
41     } temp = head;
42     printf("Linked List: ");
```

```
44     printf("Linked List: ");
45     while (temp) {
46         printf("%d -> ", temp->data);
47         temp = temp->next;
48     }
49     printf("NULL\n");
50
51     temp = head;
52     while (temp) {
53         struct Node *next = temp->next;
54         free(temp);
55         temp = next;
56     }
57
58     munmap(ptr, 200);
59
60     write(STDOUT_FILENO, "Memory released successfully.\n", 30);
61     return 0;
62 }
```

siddu@DESKTOP-5CRM05P:~/week3\$ █